



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۱۰۱۰۰۰۸۴



خلاصه شماره
۱۸

۱ خلاصه فصل چهارم - لایه شبکه (Network Layer)

در این دو ارائه، تمرکز اصلی روی کارکردهای data plane در لایه شبکه، داخل یک router، و همین طور روی مفهوم IP addressing، CIDR، پروتکل DHCP، مکانیزم NAT و در نهایت IPv6 است. سعی می کنیم در این خلاصه، همه این مفاهیم را به زبان ساده اما دقیق مرور کنیم.

۱.۱ داخل روتر چه خبر است؟ صف ها، بافرها و زمان بندی

یک router معمولاً سه بخش مهم در مسیر عبور بسته ها دارد:

□ input ports (ورودی ها)

□ switch fabric (بخش سوئیچ داخلی که بسته را از ورودی به خروجی می برد)

□ output ports (خروجی ها)

وقتی سرعت رسیدن بسته ها در هر نقطه از این مسیر بیشتر از سرعت خروج آن ها باشد، بسته ها در صف می نشینند و به تاخیر صف و گاهی ازدحام و از دست رفتن بسته می رسیم.

۱.۱.۱ صف بندی در درگاه ورودی (Input Port Queuing)

اگر switch fabric از مجموع نرخ ورودی ها کندتر باشد، بسته ها اول در input port صف می بندند. در این حالت ممکن است پدیده ای به نام blocking (HOL) Head-of-the-Line رخ دهد:

□ بسته ای که در جلوی صف قرار است به یک خروجی خاص برود، اما فعلاً امکان عبور از switch fabric را ندارد.

□ بسته های بعدی در صف، حتی اگر برای خروجی های دیگری باشند که خالی اند، پشت همین بسته گیر می کنند.

□ نتیجه: ظرفیت شبکه به خوبی استفاده نمی شود و تاخیرها بیخودی زیاد می شود.

۲.۱.۱ صفبندی در درگاه خروجی (Output Port Queuing)

در طرف دیگر، وقتی بسته‌ها از switch fabric سریع‌تر از آن‌چه لینک خروجی می‌تواند ارسال کند به output port می‌رسند، این‌بار صف در خروجی شکل می‌گیرد:

□ اگر بافر خروجی کوچک باشد، با پر شدن بافر، بسته‌های جدید دور ریخته می‌شوند.

□ این موضوع منجر به تاخیر صف و از دست رفتن بسته‌ها به خاطر سرریز بافر می‌شود.

یک قاعده تجربی در RFC 3439 می‌گوید اندازه بافر خروجی معمولاً چیزی در حد RTT متوسط ضرب در ظرفیت لینک (link capacity) است؛ یعنی تقریباً «یک RTT ترافیک» در بافر نگه داشته می‌شود تا نوسانات را صاف کند. برای تعداد زیاد جریان‌ها (flows)، پیشنهاد‌های جدیدتر می‌گویند بافر را می‌شود کوچک‌تر کرد و آن را تقریباً متناسب با $\frac{C \times RTT}{\sqrt{N}}$ گرفت که در آن N تعداد جریان‌ها است. ایده کلی این است که:

□ بافر خیلی کوچک □ خیلی زود بسته‌ها می‌ریزند.

□ بافر خیلی بزرگ □ تاخیر خیلی زیاد (bufferbloat).

۳.۱.۱ زمان‌بندی و سیاست‌های صف (Scheduling)

در هر output port وقتی چند بسته در صف هستند، یک scheduling discipline باید انتخاب کند که «کدام بسته بعدی روی لینک فرستاده شود». به زبان ساده، queue scheduler را مثل یک waiting room می‌بیند و link را مثل یک server که می‌تواند در هر لحظه فقط یک بسته را بفرستد.

سیاست‌های ساده زمان‌بندی

□ FIFO (اول وارد، اول خارج): بسته‌ها دقیقاً به ترتیب ورودشان ارسال می‌شوند. این ساده‌ترین و رایج‌ترین سیاست است.

در کنار سیاست زمان‌بندی، یک سیاست دور ریختن (discard policy) هم داریم برای وقتی بافر پر شده است:

□ tail drop: بسته جدیدی که می‌رسد و جا نیست را به سادگی دور می‌ریزیم.

□ priority drop: بسته‌ها را بر اساس اولویت نگاه می‌کنیم و در صورت نیاز، بسته‌های با اولویت پایین را حذف می‌کنیم.

□ حذف تصادفی (random drop): به صورت تصادفی از میان بسته‌های صف چندتا را حذف می‌کنیم (ایده مشابه RED برای جلوگیری از ازدحام ناگهانی).

زمان بندی اولویتی (Priority Scheduling) در priority scheduling، بسته ها در چند کلاس مختلف با اولویتهای متفاوت قرار می گیرند و همواره بسته در حال خدمت از کلاس با اولویت بالاتر انتخاب می شود:

□ کلاس ها می توانند بر اساس فیلدهای هدر IP مثل آدرس مبدأ/مقصد، شماره پورت، یا Type of Service تعریف شوند.

□ مثلاً ترافیک real-time (مثل صدا و ویدئو) می تواند در کلاس با اولویت بالاتر باشد و ترافیک bulk transfer (دانلودها) در کلاس با اولویت پایین تر.

این مدل، کیفیت سرویس (QoS) را برای برخی جریان ها بهتر می کند، ولی بحث های جدی درباره network neutrality ایجاد می کند که آیا منصفانه است بعضی ترافیک ها همیشه وضعیت بهتری داشته باشند؟

زمان بندی Round Robin و Weighted Fair Queuing برای اینکه هم عدالت رعایت شود و هم کلاس های مختلف سهمی از ظرفیت لینک داشته باشند، از سیاست های پیشرفته تری استفاده می شود:

□ Round Robin (RR): چند queue مجزا برای کلاس های مختلف داریم و به شکل چرخشی از هر صف یک بسته می فرستیم (اگر چیزی در آن صف باشد). این کار باعث می شود هیچ کلاسی کاملاً گرسنه نماند.

□ Weighted Fair Queuing (WFQ): نسخه تعمیم یافته RR است که در آن برای هر کلاس یک وزن در نظر گرفته می شود؛ در هر چرخه به نسبت وزن ها، ظرفیت لینک بین کلاس ها تقسیم می شود. مثلاً اگر یک کلاس وزن دو و دیگری وزن یک داشته باشد، تقریباً دو برابر ترافیک کلاس اول نسبت به دومی ارسال می شود.

این سیاست ها کمک می کنند سرویس های حساس (مثل VoIP، video streaming) و سرویس های معمولی (مثل وب گردی و دانلود) همزمان و با کیفیت قابل قبول روی یک لینک مشترک کار کنند.

۲.۱ ادامه لایه شبکه: IP، آدرس دهی و تکه تکه شدن (Fragmentation)

هر بسته IP که به آن datagram هم می گوئیم، یک هدر دارد (معمولاً ۲۰ بایت برای IPv4) که اطلاعاتی مانند آدرس مبدأ و مقصد، طول کل، و شناسه تکه ها را نگه می دارد. اگر روی TCP هم کار کنیم، ۲۰ بایت هدر TCP را هم باید در نظر بگیریم؛ یعنی حدود ۴۰ بایت سربار فقط برای IP و TCP، بدون احتساب لایه کاربرد.

۱.۲.۱ حداکثر اندازه قاب (MTU) و تکه تکه شدن بسته

هر لینک در شبکه یک MTU (Maximum Transfer Unit) دارد؛ یعنی حداکثر اندازه ای که یک frame در آن لینک می تواند داشته باشد. اگر یک IP datagram بزرگ تر از MTU باشد:

□ روتر وسط راه آن را به چند تکه (fragment) می شکند.

□ هر تکه یک هدر IP جداگانه دارد که در آن شناسه و offset (محل قطعه در بسته اصلی) نوشته می شود.

□ در مقصد نهایی، این تکه ها دوباره با هم ترکیب می شوند و بسته اصلی بازسازی می شود.

این فرآیند fragmentation و reassembly است. هرچه شبکه ناهمگن تر باشد و MTUها متفاوت، احتمال تکه تکه شدن بیشتر است و سربار پردازشی در مقصد بالا می رود؛ به همین دلیل معمولاً سعی می شود سائز بسته اولیه را طوری انتخاب کنند که روی مسیرهای معمول نیاز زیادی به fragmentation نباشد.

۳.۱ IP Addressing و CIDR: آدرس دهی مدرن در اینترنت

۱.۳.۱ آدرس IP و مفهوم interface

آدرس IP در IPv4 یک شناسه ^{۳۲} بیتی است که به *interface* نسبت داده می شود، نه به خود میزبان به صورت کلی. هر interface روی یک لینک فیزیکی (مثل Ethernet، Wi-Fi) یک آدرس IP دارد:

□ یک router معمولاً چند interface دارد؛ پس چند آدرس IP.

□ یک میزبان خانگی ممکن است یک Ethernet و یک Wi-Fi داشته باشد (یعنی دو interface و دو آدرس IP).

۲.۳.۱ Classless InterDomain Routing - CIDR

در گذشته آدرس ها به صورت classful (کلاس A، B، C) تقسیم می شدند، اما این روش انعطاف کمی داشت. CIDR این محدودیت را از بین می برد:

□ فرم کلی آدرس: $a.b.c.d/x$ که در آن x تعداد بیت های بخش شبکه است.

□ بقیه بیت ها، بخش میزبان (host part) را تشکیل می دهند.

مثلاً $200.23.16.0/23$ یعنی ^{۲۳} بیت اول مربوط به «شبکه» است و ^۹ بیت بعدی برای میزبان هاست. این جوری شبکه ها می توانند در اندازه های مختلف (کوچک یا بزرگ) ساخته شوند و routing table هم می توانند ورودی های کلی تری داشته باشند.

۳.۳.۱ سابنت‌ها (Subnets)

برای یک سازمان معمولاً آدرس بزرگ‌تری گرفته می‌شود و بعد خود سازمان آن را به چند subnet می‌شکنند:

□ دستگاه‌هایی که بخش شبکه آدرس‌شان (بیت‌های بالا) یکی است و بدون روتر به هم وصل‌اند، یک subnet را تشکیل می‌دهند.

□ یک subnet mask مثل 24/ یعنی ۲۴ بیت اول برای شبکه و ۸ بیت آخر برای میزبان.

برای تشخیص تعداد subnetها، می‌توان همه interfaceها را از میزبان‌ها جدا کرد و هر تکه متصل مستقل را یک subnet در نظر گرفت.

۴.۱ چطور یک IP address می‌گیریم؟

این سوال دو بخش دارد:

۱. میزبان (مثلاً لپ‌تاپ) آدرس خودش را چطور می‌گیرد؟

۲. خود شبکه (مثلاً شبکه دانشگاه یا شرکت) بلوک آدرس را از کجا می‌گیرد؟

۱.۴.۱ آدرس گرفتن میزبان: DHCP

میزبان می‌تواند:

□ آدرس را به صورت دستی در فایل تنظیمات سیستم (مثلاً /etc/rc.config) تنظیم کرده باشد، یا

□ از پروتکل DHCP (Dynamic Host Configuration Protocol) استفاده کند.

DHCP یک پروتکل ۰۰۰۰۰-۰۰۰۰-۰۰۰۰ است؛ یعنی وقتی لپ‌تاپ را به شبکه وصل می‌کنید، خودش اتوماتیک آدرس می‌گیرد. روند تقریبی به این صورت است:

۱. DHCP Discover: میزبان یک پیام broadcast روی شبکه می‌فرستد (src: 0.0.0.0, dest: 255.255.255.255) تا بپرسد «اینجا سرور DHCP هست؟».

۲. DHCP Offer: سرور DHCP (که اغلب در یک router تعبیه شده) جواب می‌دهد و یک آدرس IP پیشنهادی، مدت اعتبار (lease time) و اطلاعات دیگر را اعلام می‌کند.

۳. DHCP Request: میزبان اعلام می‌کند که همان آدرس را می‌خواهد.

۴. DHCP ACK: سرور تایید می کند و از این لحظه به بعد، آن آدرس به میزبان «اجاره» داده شده است.

DHCP فقط آدرس IP نمی دهد؛ معمولاً همراهش:

□ آدرس روتر پیش فرض (default gateway یا همان first-hop router)،

□ آدرس سرور DNS،

□ و subnet mask

را هم می فرستد. یعنی بعد از DHCP، میزبان عملاً همه چیز را برای صحبت با اینترنت می داند.

۲.۴.۱ آدرس گرفتن شبکه از ISP

خود شبکه سازمانی بلوک آدرس را از ISP می گیرد. ISP هم یک بلوک بزرگتر از یک مرجع بالاتر می گیرد و آن را به بلوک های کوچکتر تقسیم و به سازمان ها می دهد. مثلاً اگر ISP بلوک 200.23.16.0/20 را داشته باشد، می تواند آن را به چند بلوک 23/ مثل 200.23.16.0/23، 200.23.18.0/23 و ... تقسیم و به سازمان ها اختصاص دهد. این کار باعث:

□ مدیریت راحت تر آدرس ها،

□ و امکان route aggregation در اینترنت می شود؛ یعنی ISP می تواند به بقیه دنیا بگوید: «هر چه آدرسی با پیشوند 200.23.16.0/20 بود را به سمت من بفرستید» و لازم نیست برای هر سازمان یک ورودی جدا تبلیغ کند.

وقتی یک سازمان از یک ISP به ISP دیگری مهاجرت کند، ISP جدید می تواند برای آن سازمان یک مسیر خاص تر (more specific route) مثل 200.23.18.0/23 را تبلیغ کند تا ترافیک آن سازمان مستقیماً به ISP جدید برود. در نهایت، نهاد ICANN (Internet Corporation for Assigned Names and Numbers) در سطح دنیا مدیریت ریشه نام ها و بلوک های بزرگ آدرس را بر عهده دارد و آن ها را به regional registries مختلف تخصیص می دهد.

۵.۱ NAT: ترجمه آدرس شبکه ای

۱.۵.۱ انگیزه استفاده از NAT

با توجه به محدود بودن فضای آدرس های IPv4، و همچنین نیاز به پنهان سازی ساختار داخلی شبکه، مکانیزم NAT (Network Address Translation) به شدت رایج شد. ایده اصلی:

□ در داخل شبکه محلی (مثلاً خانه) دستگاه‌ها از آدرس‌های خصوصی مثل 10.0.0.x یا 192.168.x.y استفاده می‌کنند.

□ در بیرون، همه این دستگاه‌ها با یک آدرس عمومی مشترک دیده می‌شوند؛ مثلاً 138.76.29.7.

مزایا:

□ صرفه‌جویی در آدرس‌ها: فقط یک آدرس عمومی از ISP لازم است.

□ انعطاف در تغییر داخلی: می‌توان آدرس‌های داخلی را عوض کرد بدون اینکه بیرون چیزی بداند.

□ نوعی محافظت امنیتی: دستگاه‌های داخلی مستقیماً از بیرون قابل آدرس‌دهی نیستند؛ باید از طریق NAT عبور کنند.

۲.۵.۱ چطور کار می‌کند؟

یک NAT router یک جدول ترجمه نگه می‌دارد:

□ برای هر اتصال خروجی، جفت (private IP, private port) را به یک جفت (public NAT IP, new port) نگاشت می‌دهد.

□ در بسته‌های خروجی، آدرس و پورت مبدأ را با این جفت عمومی جایگزین می‌کند.

□ وقتی پاسخ از بیرون می‌آید (با مقصد (NAT IP, new port))، NAT با نگاه به جدول ترجمه، آن را دوباره به (private IP, private port) برمی‌گرداند و به دستگاه داخلی می‌فرستد.

چون فیلد پورت در UDP/TCP ۱۶ بیتی است، تئوریکاً تا حدود 60,000 اتصال همزمان می‌توان با یک آدرس عمومی داشت (تا وقتی پورت‌های منحصربه‌فرد اختصاص داده شود).

۳.۵.۱ چالش‌ها و بحث‌ها

NAT هم مفید است و هم بحث‌برانگیز:

□ از دید اصول کلاسیک شبکه، روترها باید فقط تا لایه ۳ (IP) را دست‌کاری کنند، اما NAT مجبور است فیلدهای لایه ۴ (پورت‌ها) را هم عوض کند؛ این کار اصل انتها به انتها (end-to-end principle) را نقض می‌کند.

□ اگر کلاینت بیرونی بخواهد خودش به یک سرور داخل شبکه پشت NAT وصل شود، کار سخت می‌شود و نیاز به روش‌هایی مثل NAT traversal، port forwarding، یا پروتکل‌های کمکی (UPnP، STUN، ...) داریم.

□ با وجود این، در شبکه های خانگی و 4G/5G و بسیاری از شبکه های سازمانی، NAT عملاً استاندارد شده و کنار گذاشتن آن سخت است.

۶.۱ IPv6: نسل بعدی آدرس دهی اینترنت

۱.۶.۱ چرا IPv6 لازم شد؟

دو انگیزه اصلی:

□ فضای ۳۲ بیتی IPv4 تقریباً تمام شده بود و ICANN آخرین بلوک های بزرگ را سال ها پیش تخصیص داد.

□ فرصت خوبی بود که هدر IP ساده تر و برای QoS و مفهوم flow بهینه تر شود.

IPv6 آدرس ها را ۱۲۸ بیتی کرده است؛ یعنی فضای آدرس تقریباً نامحدود به نظر می رسد. با این حال، مهاجرت کامل به IPv6 کند بوده و سال ها است که دنیا در حال گذار تدریجی است و هر دو نسخه (IPv6 و IPv4) کنار هم استفاده می شوند.

۲.۶.۱ فرمت بسته IPv6

header IPv6 به صورت ثابت و ۴۰ بایتی طراحی شده و فیلدهای اصلی آن عبارتند از:

□ version: نسخه پروتکل (در اینجا ۶).

□ priority یا traffic class: برای تعیین اولویت جریان ها.

□ flow label: برای مشخص کردن اینکه این بسته متعلق به کدام جریان (flow) است.

□ payload length: طول داده بعد از هدر.

□ next header: پروتکل لایه بالاتر (مثلاً TCP، UDP یا هدرهای افزونه ای extension headers).

□ hop limit: مثل TTL در IPv4، تعداد گام های باقی مانده.

□ source address و destination address: هر کدام ۱۲۸ بیت.

چند نکته مهم:

□ در IPv6 دیگر checksum در هدر وجود ندارد؛ این باعث سریع تر شدن پردازش در روترها می شود (چون لازم نیست هر روتر دوباره checksum را حساب کند).

□ گزینه‌ها (options) به extension headers منتقل شده‌اند تا هدر اصلی ثابت و کوچک بماند.

به طور خلاصه، IPv6 قرار است هم مشکل کمبود آدرس را حل کند، هم با هدر ساده‌تر، روترها را سریع‌تر کند و هم زمینه را برای کیفیت سرویس بهتر و مدیریت جریان‌ها فراهم کند؛ هرچند در عمل هنوز ترکیب NAT + IPv4 + شروع تدریجی IPv6 واقعیت امروز اینترنت است.

مراجع